



Introducción

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

Un día normal...



¿Cómo lo afrontan?

¿Qué preguntas se hacen ustedes?

¡A la carga!

El enemigo que actúa
aisladamente, que carece de
estrategia y que toma a la
ligera a sus adversarios,
inevitablemente acabará
siendo derrotado

Sun Tzu, "El arte de la guerra"
Hace tiiieempo





Sobre su diseño inicial

- Qué pasa si...
 - ¿Agregamos más clientes?
 - ¿Se caen los servidores?
 - ¿Hay un bug en producción?
 - ¿Si hay que agregar una *feature* a futuro?
 - ¿Modifican el presupuesto?
 - ¿Compramos otra empresa?
 - ¿Agregamos un colaborador o cien colaboradores?





¿Quiénes están involucrados hoy en los sistemas empresariales?

<i>Cliente</i>	<i>Tech</i>	<i>Management</i>
Clientes	Desarrolladores	Sales
Usuarios	QA	Project managers
Dueños	DevOps	C-Level
Terceros	Agile enablers	HR

¿Quiénes están involucrados hoy en los sistemas OSS?

<i>Cliente</i>	<i>Tech</i>	<i>Owners</i>
Clientes	Desarrolladores	Líderes
Usuarios	Colaboradores	SIGs
Support a terceros	Audidores	Empresas del rubro
Integradores	Testers	HR

Frentes de presión para el software



CALIDAD



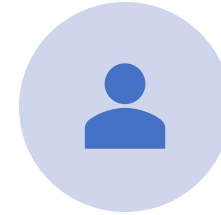
TIEMPO



DINERO



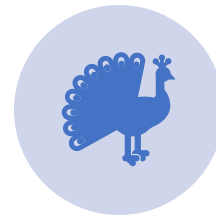
ESFUERZO



PERSONAL



MERCADO



ELEGANCIA...



Y llega la arquitectura...



¿Qué es la Arquitectura de Software?

“Es un set de **estructuras** requeridas para razonar acerca de un sistema, y comprende **elementos** de software, **relaciones** entre ellas y **propiedades** de ambas”

(Bass, Kazman, 2013)

Más definiciones

“Software architecture is about the **important** stuff, whatever it is”

(Ralph Johnson, coautor del GoF)

“There are two common elements: **Highest-level** breakdown of a system into its parts; the other, decisions that are **hard to change**”

(Martin Fowler, Patterns of Enterprise architecture)

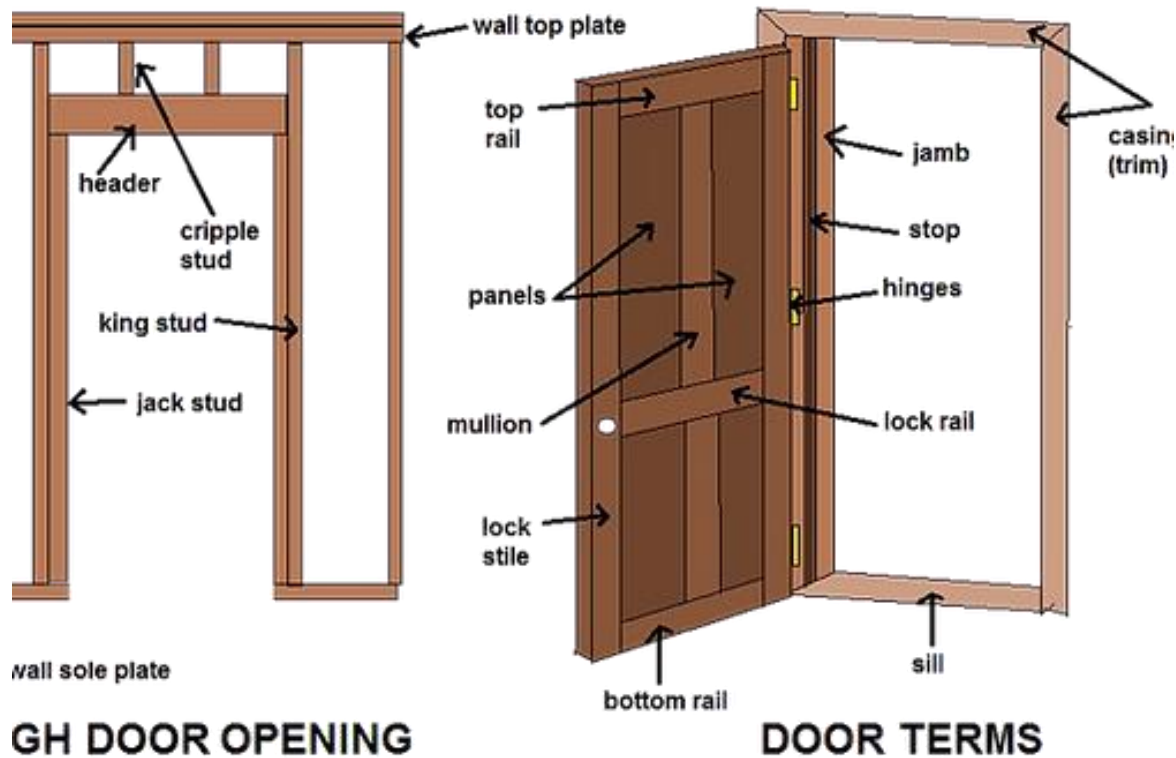
“Software architecture is the *stuff* that’s **hard to change later**”

(Neal Ford, Software Architecture: The Hard Parts)



Analogía de arquitectura en la construcción

¿Por qué tienen esas características?





Comparación más directa

Construcción	Software
Un arquitecto guía el proyecto desde un punto de vista holístico	
No basta con que el arquitecto sepa construir, requiere conocimiento y experiencia	No basta con saber programar, se requiere conocimiento adicional
Requiere comunicarse con un equipo de trabajo amplio (ingenieros, constructores, administradores, dueños, etc)	Requiere comunicarse con un equipo de trabajo amplio: Ingenieros, PM, PO, QA, Cliente, usuario, Dueños, C-level
Capacidad de hacer mock-ups	Capacidad de lograr PoCs y planificaciones
Utiliza patrones de construcción y los incorpora al diseño	Utiliza patrones y estilos de software ampliamente probados y usados
Reconoce anti-patrones y reduce posibilidad de errores, tiene puntos de revisión	
Se enfoca en atender necesidades del futuro usuario	

Si salimos de la analogía

Construcción	Software
Cambiar post-construcción realmente es difícil	Hay un costo menor y se pueden hacer grandes migraciones, tiene un aspecto temporal
Costo basado en CAPEX (lo que se comprará)	Costo basado en OPEX (lo que se gastará)
Despliegue difícilmente escalable	Más fácil de escalar, especialmente si se intenciona
Industria estructurada, antigua, con muchos estándares asociados	Industria joven y menos estructurada
Más fácil medir calidad y resultados directo a la vista	Medir calidad es todo un desafío y no es aparente de inmediato
Utiliza diversos niveles de calificación en mano de obra	Colaboradores especializados

Grandes líneas de la arquitectura

Representa la organización del sistema y del negocio

“La organización fundamental de un sistema se ve reflejada en sus **componentes**, relaciones entre ellos y el ambiente, y los **principios** que gobiernan su diseño y evolución.”

*(Recommended Practice for Architectural Description of
Software-Intensive Systems, ANSI/IEEE Std 1471-2000)*

Grandes líneas de la arquitectura

No es solo diseñar, es tomar decisiones usualmente difíciles

“All **architecture is design**, but not all design is architecture. Architecture represents the significant design **decisions** that shape a system, where significant is measured by **cost** of change”

(Grady Booch (UML, OOP y otros), 2006)

¿Entonces quién es el arquitecto?

- Elemento central
- Responsable directo
- Multifacético con enfoque técnico
- Puede estar apoyado por un sub-equipo



Sobre el arquitecto

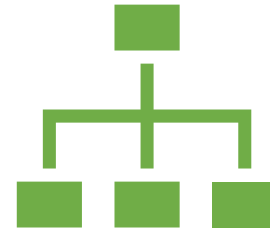


Las mejores arquitecturas son producto de:

Solo una “mente”

Un equipo pequeño y estructurado

- *Rechtin, Systems Architecting: Creating & Building Complex Systems, 1991, p21*



Todo proyecto debe tener exactamente 1 arquitecto identificable (1 responsable):

Para proyectos más grandes, el arquitecto principal debe/puede estar respaldado por un arquitecto por sub-equipo

- *Booch, Object Solutions, 1996*

¿Qué hace?

- Toma decisiones de grano grueso
- Liga requerimientos a propuestas arquitectónicas
- Piensa en **trade-offs**
- Mantiene a todos los involucrados en su línea de visión
- También **programa**, no vive en torre de marfil.

“Toma decisiones arquitectónicamente **significativas**, que afectan la estructura, RNFs, dependencias, interfaces o técnicas de construcción”

(Michel Nygard, Release It!)

¿Qué debe saber?

- Conoce una **variedad de sistemas y posibilidades**
- Tiene conocimiento de dominio
- Es capaz de **modelar y amoldar soluciones**
 - Escalar
 - Evolucionar
- Entiende el problema subyacente y que se quiere solucionar

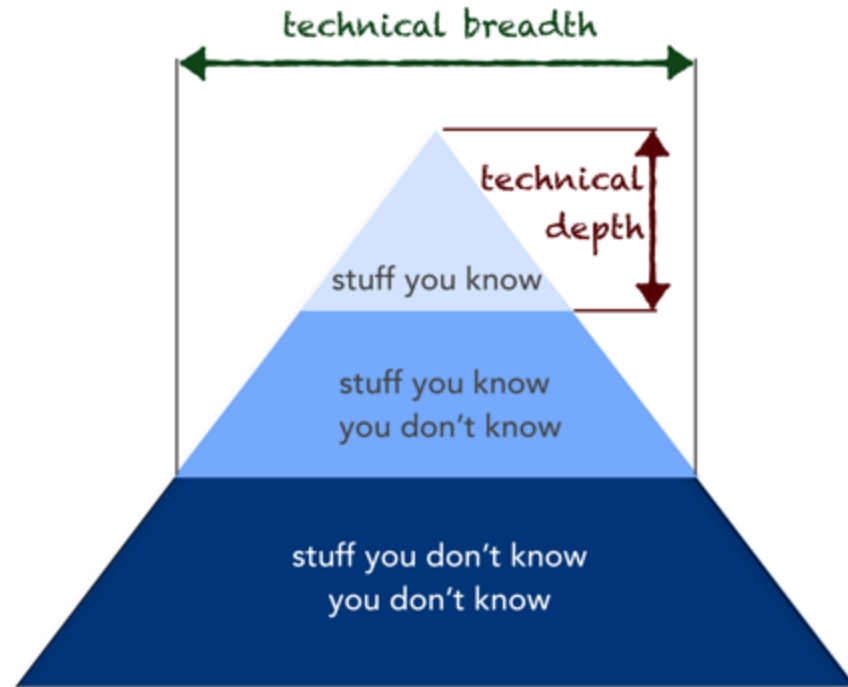


¿Qué debe saber?

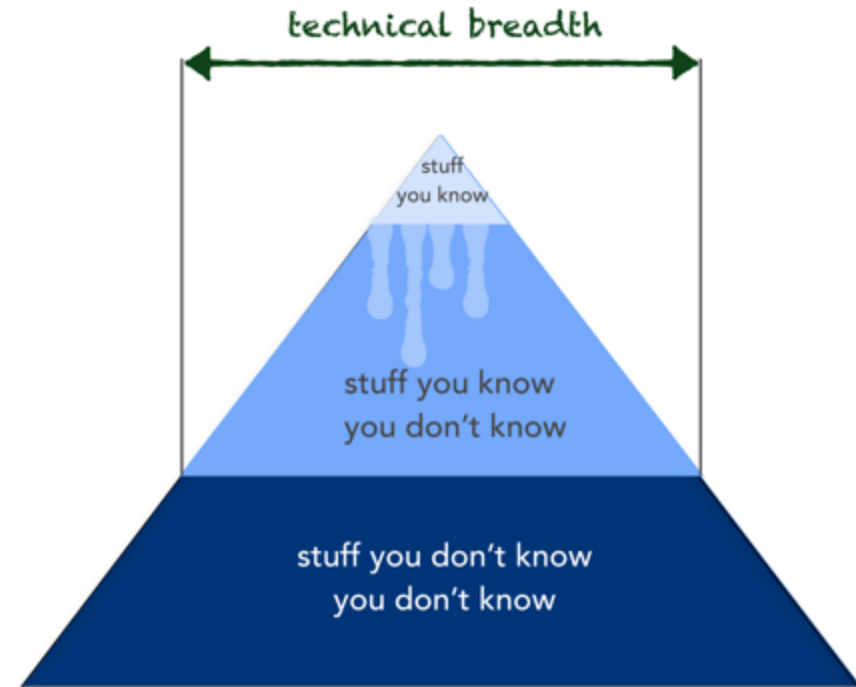
- Debe **estar al día con lo que ocurre en el mundo tech**
- No tiene remordimientos por cambiar a un paradigma superior
 - Siempre y cuando estuviera bien justificado...
- Es **capaz de demostrar expertise**
 - Mediante PoCs
 - Posee una o más áreas profundas



¿Cómo “sabe” el arquitecto?



Developer



Arquitecto

Queremos expandir lo que nosotros sabemos que no sabemos

El arquitecto debe proyectar...

- Tomar **decisiones de largo plazo**, conociendo el costo (no solo monetario)
- Observar todos los stakeholders, **propone y decide**
- Comprender procesos de negocio y particularidades
- Observar la competencia



El arquitecto debe hablar...

- Conversa con todas las partes
 - Escuchar, negociar, explicar, convencer, vender
- Coordina requisitos y capacidades
- Explica problemas
 - Con calma y respeto
 - Por qué pasa esto o esto



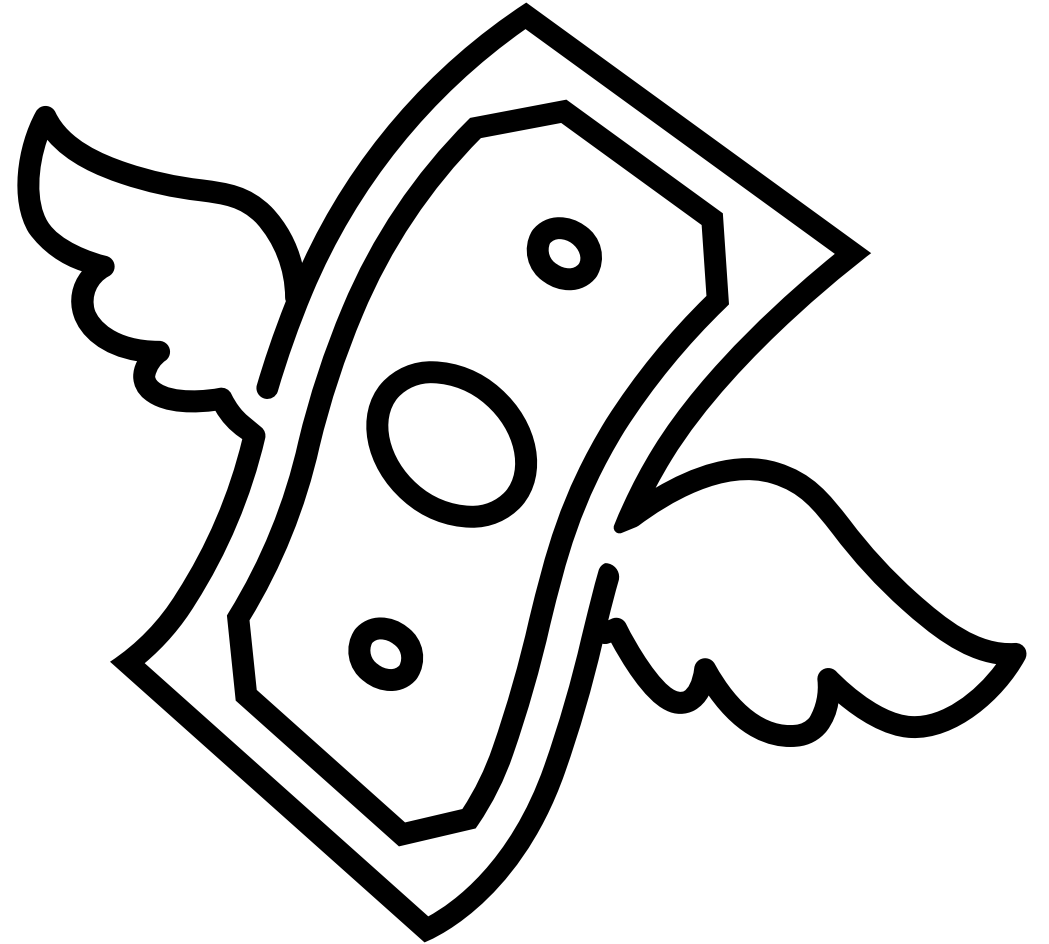
El arquitecto debe liderar...

- Posee un **pie técnico** que le respalda frente a:
 - Desarrolladores
 - Equipos
 - Otros líderes
- Posee un **pie de negocio** que le respalda frente a:
 - Presupuestos
 - Product Owners
 - Usuarios
- Inspira respeto e impulsa a que **se sigan las guías y decisiones** que justifica
- **No impone**, convence y guía
- Lidia con la política empresarial



El arquitecto debe estimar...

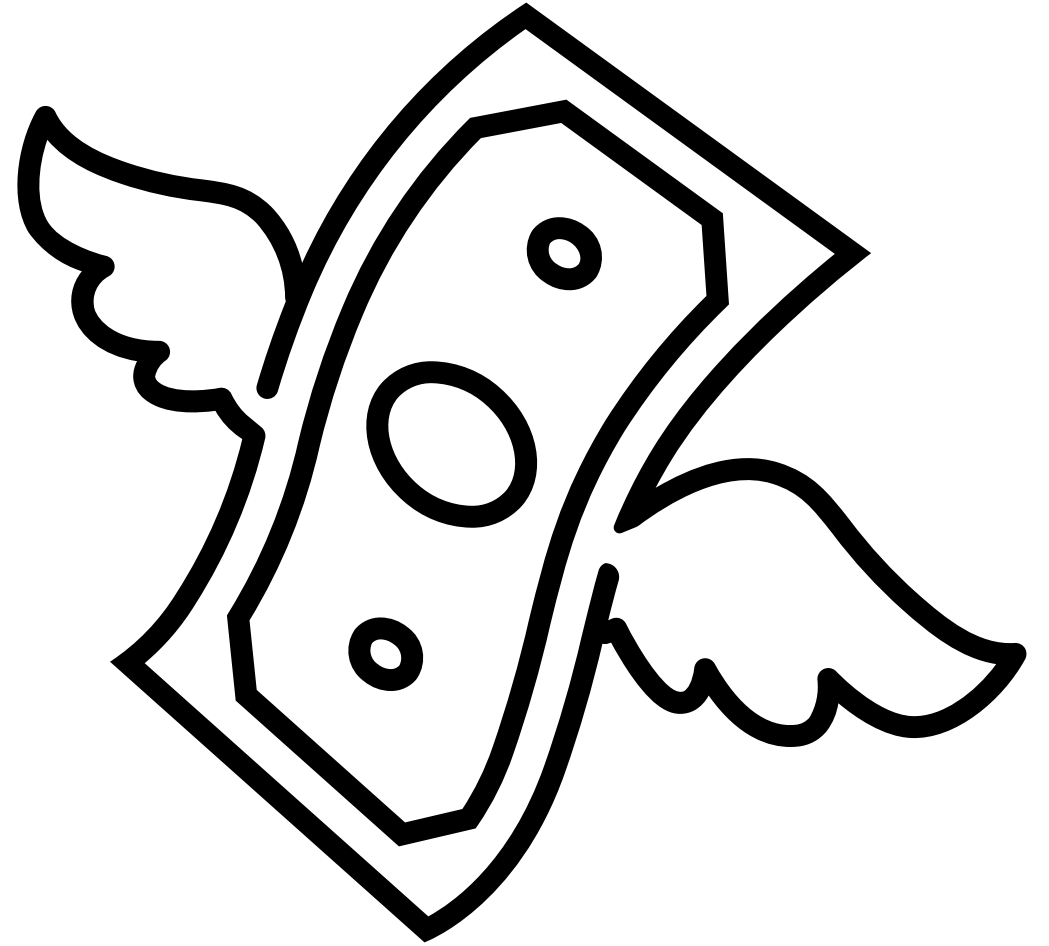
- **Observa costos**
 - Software COTS (no desde cero)
 - Por desarrollar
 - Integrabilidad y flexibilidad
 - FinOps
- No siempre se puede usar todo lo que se desea
- Puede aproximar costos y lograr decisiones generales



—

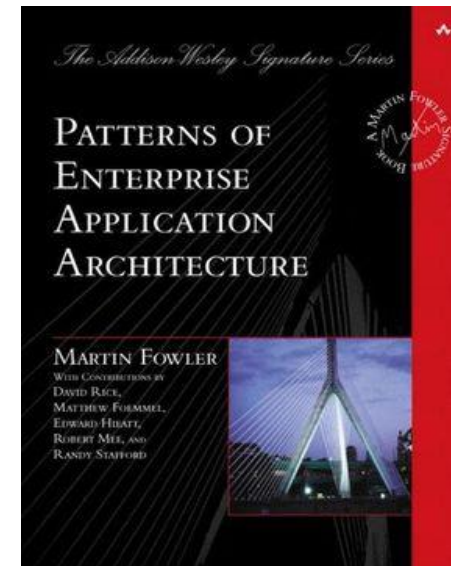
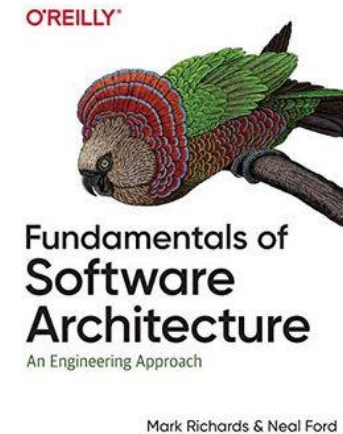
¿Esto me sirve aunque no sea arquitecto mañana?

- La demanda de habilidades va por la línea de análisis y conexión con problemas
- Estrategias sobre implementaciones
- El valor de la modelación



Bibliografía

- Mark Richards, Neal Ford: "Fundamentals of Software Architecture, An engineering approach"
- Martin Fowler et al.: "Patterns of Enterprise Application Architecture"
- Bass, Kazman: "Software architecture in practice"



Sobre el curso (meta)



Aprenderán a diseñar sistemas considerando aspectos de dominio



Utilizarán herramientas y tecnología presentes en la industria



Podrán modelar sistemas en base a requerimientos ambiguos

Plan general del curso

Proyecto

Lenguaje común
Definiciones base
Conceptos esenciales
NFR

Estrategias
Diagramación
Estilos
Patrones
Interfaces

Aplicaciones
Performance
Seguridad
Devops
Patrones avanzados

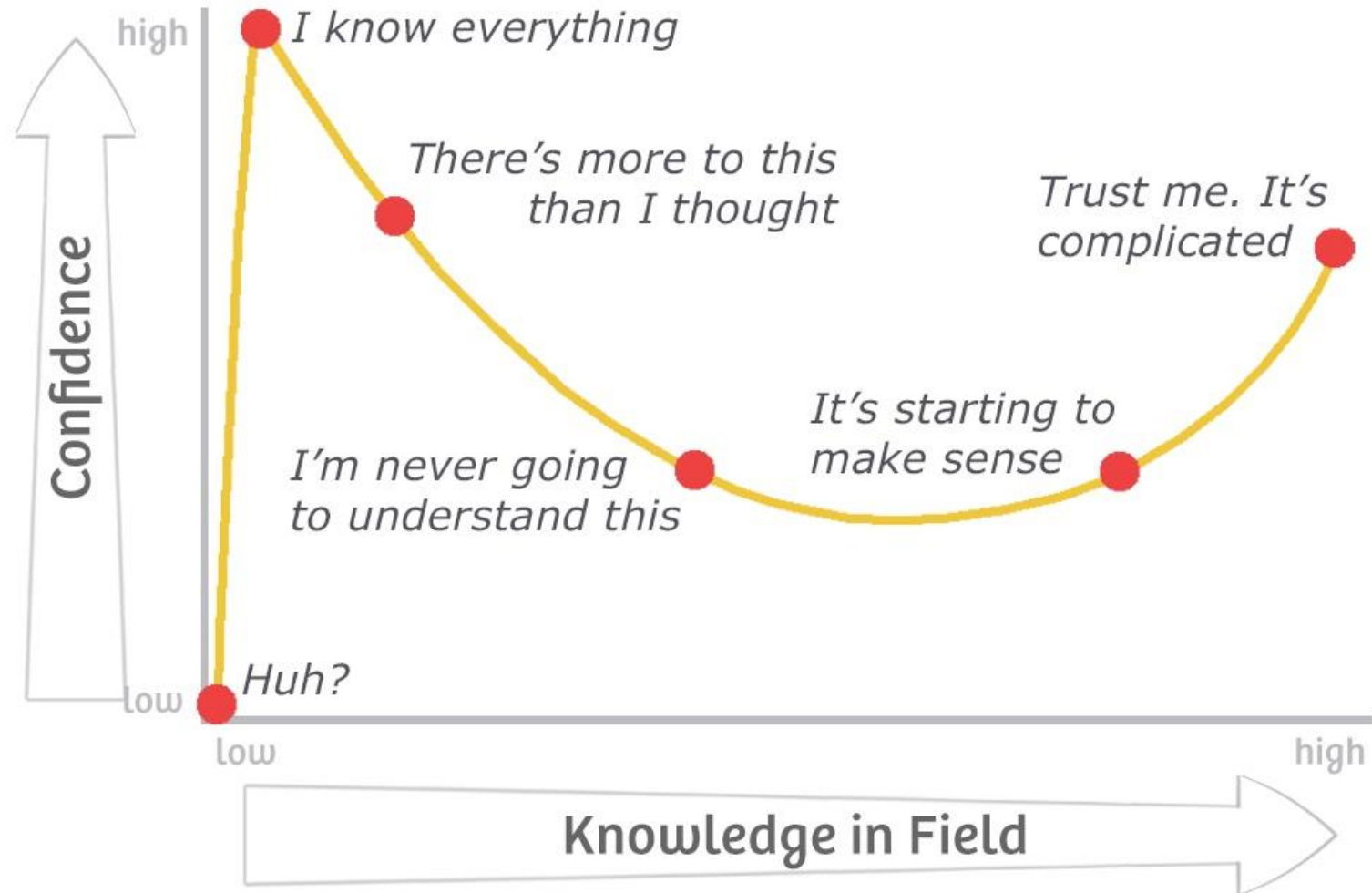
Cátedra

Cloud computing

Modelacion

Software COTS
Integrabilidad

Sobre el curso



Principios que guían a este curso



Exploración complementaria
autónoma



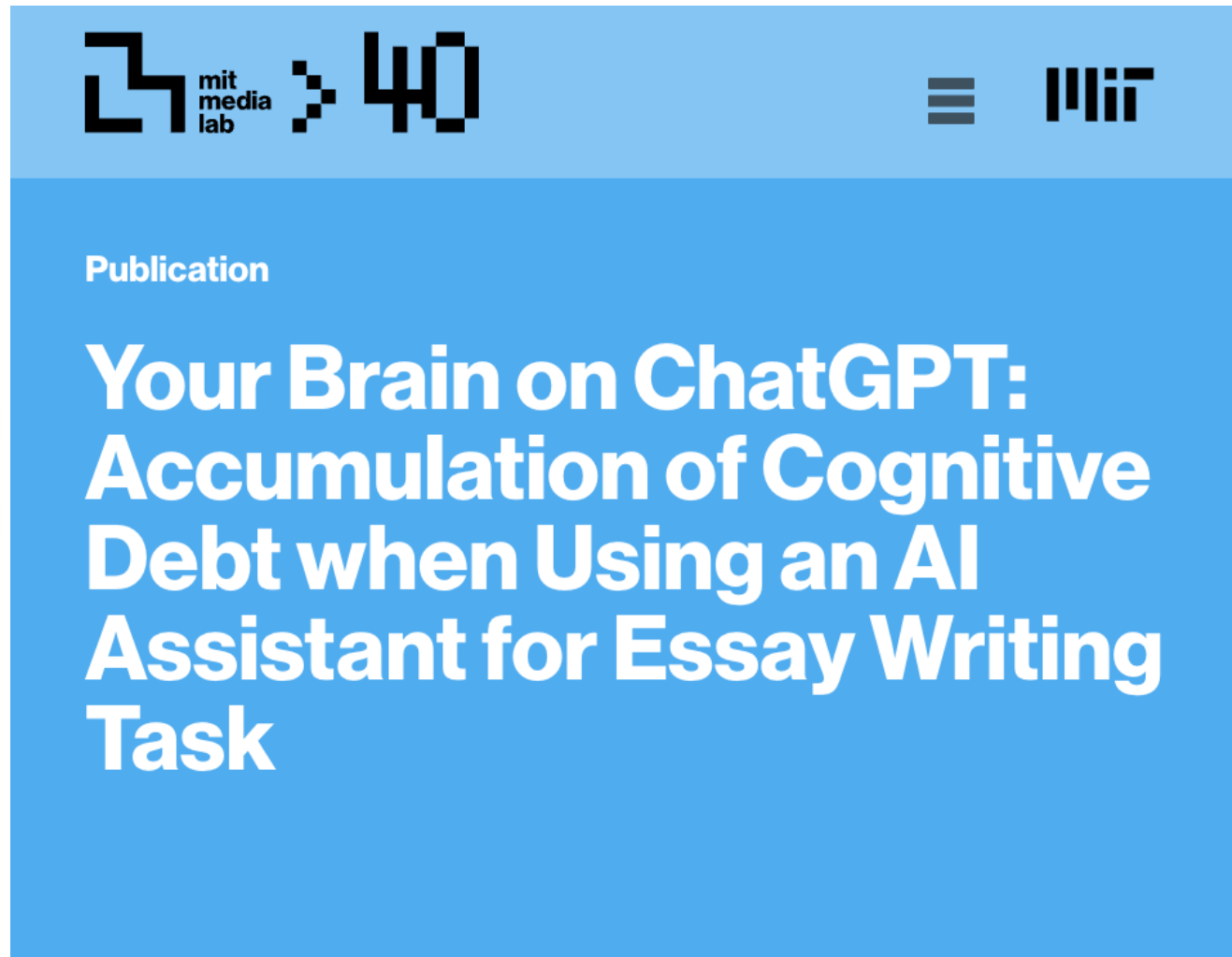
Flexibilidad evaluativa
Pero no free-riders



Acompañamiento en el
aprendizaje

Sobre el uso de AI

- Detrimental en el proceso inicial de aprendizaje
 - Muchos conceptos nuevos
 - Conexiones entre conceptos nuevos y antiguos
 - Brain economy?
- Bordes en uso
 - Proyecto: Libre en Código
 - Interrogaciones: Solo resumir o consulta de conceptos
 - Actividades: No permitido
 - Casos de debates: No permitido







Modelar sistemas complejos con tradeoffs



Debatir decisiones frente a pares y stakeholders



Que esperamos de ustedes



Capacidad de buscar información
y discernir cuándo pueden usar
las IAs

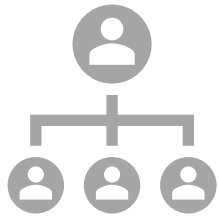


Participación en clases y en
instancias extra-clases



Comunicación abierta y feedback
constructivo

Tres tipos de evaluaciones



Actividades



Pruebas



Proyecto

Actividades

Enfoque en que puedan pensar rápidamente y defender soluciones

- Grupos adhoc
- Se elimina el 30% de los quizzes
- Se elimina la peor modelación

Instancia	Tipo	Peso
Quiz	Individual	33%
Modelaciones	Grupal ad-hoc	33%
Debate	Grupal ad-hoc	34%

Pruebas

Enfoque en que puedan justificar decisiones

- 2 Interrogaciones + examen
- Examen eximible y elimina la peor nota
- Ies online con 4 días para elaborar

Instancia	Peso
I1	33%
I2	33%
EX	34%

$$NIE = (\text{SUM}(I1, I2, 2 * Ex) - \text{Min}(I1, I2, Ex))/3$$

Proyecto

Enfoque en que puedan ir descubriendo nuevas tecnologías y capacidades de investigación

- E0 individual, las siguientes grupales
- Sujeto a evaluación de pares **total**
- Pueden ser grupos intersecciones
- Acumulativas
- No habrá instancias recuperativas

Instancia	Tipo	Peso
E0	Individual	25%
E1	Grupal	20%
E2	Grupal	25%
E3	Grupal	30%

$$N_i = 1 + N_{ig} * (F_i - 1)$$

Aprobación

- Deben cumplir todos los criterios y responder un formulario de conocimientos que se les enviará
- **Este curso adscribe a la política de integridad académica del DCC y al código de honor**
- Confiamos en que ya poseen una ética profesional en buen desarrollo

$$0.4 NP + 0.3 NA + 0.3 NIE = NF$$

$$NIE \geq 3.95$$

$$NE \geq 3.00$$

$$NA \geq 3.95$$

$$NP \geq 4.00$$

$$EPP^* \geq 0.7$$

$$NF \geq 3.95$$

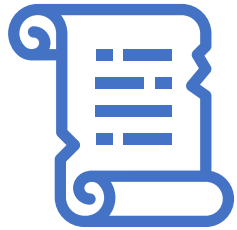
Actividad 0

Diseñen alguno de los siguientes sistemas

Uber



Actividad 0



Entreguen todos los artefactos
que consideren importantes para
exponer su solución



Expliquen el por qué lo diseñan
así



Pueden entregarlo en papel u
online



Definiciones clave

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

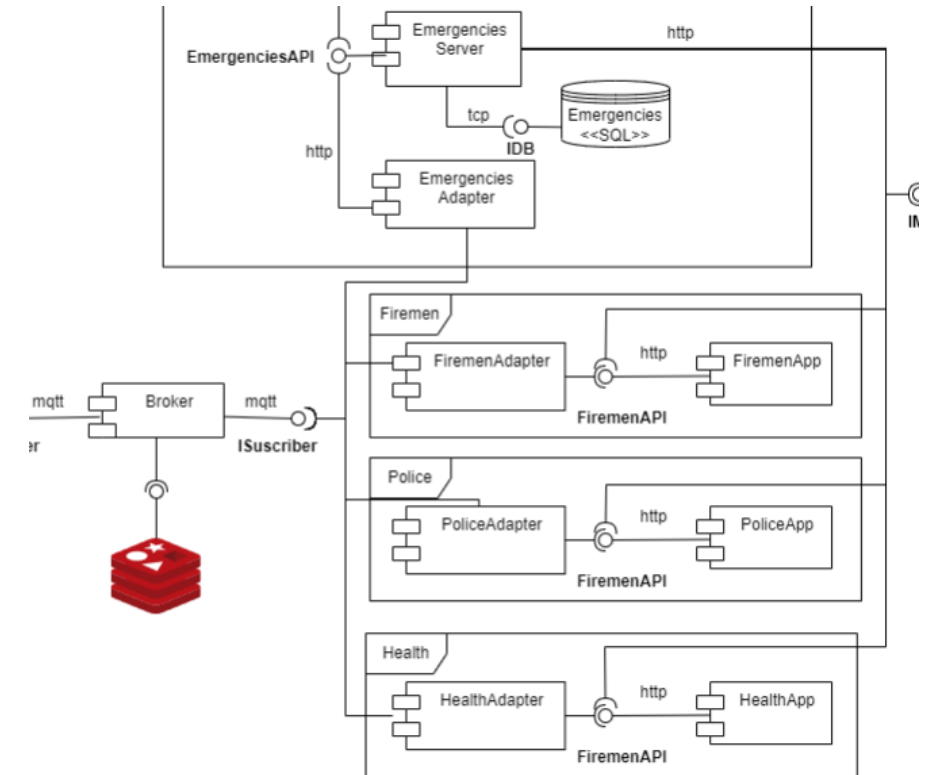


Facts clave de la arquitectura de sistemas

- “Everything in software architecture is a **trade-off**”
 - Muchas de las decisiones tomadas pueden llegar a ser sub-óptimas
- “**Why** is more important than how”
 - No podemos tomar decisiones únicamente porque nos son más fáciles
- “Architecture is the **stuff you can't Google**”
 - Mark Richards – Fundamentals of Software Architecture
 - Muchas veces surgen cosas no-googleables
- “Any organization that designs a system (defined broadly) will produce a **design whose structure is a copy of the organization's communication structure**”
 - Melvin E. Conway – Ley de Conway

¿Dónde enfoca el ojo arquitectónico?

- Pensar en *trade-offs*
- Mirar más allá de las clases y líneas de código
- Encontrar lo mejor al caso, no necesariamente perfección
- Agregar y desagregar el Sistema



¿Cómo aplicamos esto a los casos anteriores?

Se debe tener: Conocimiento de dominio

Capacidad de hablar y reconocer problemas de forma general en un campo de conocimiento específico

No queremos saber sólo de las tecnologías (conocimiento técnico), **también necesitamos del campo del negocio**

Requisito para lograr un diseño exitoso

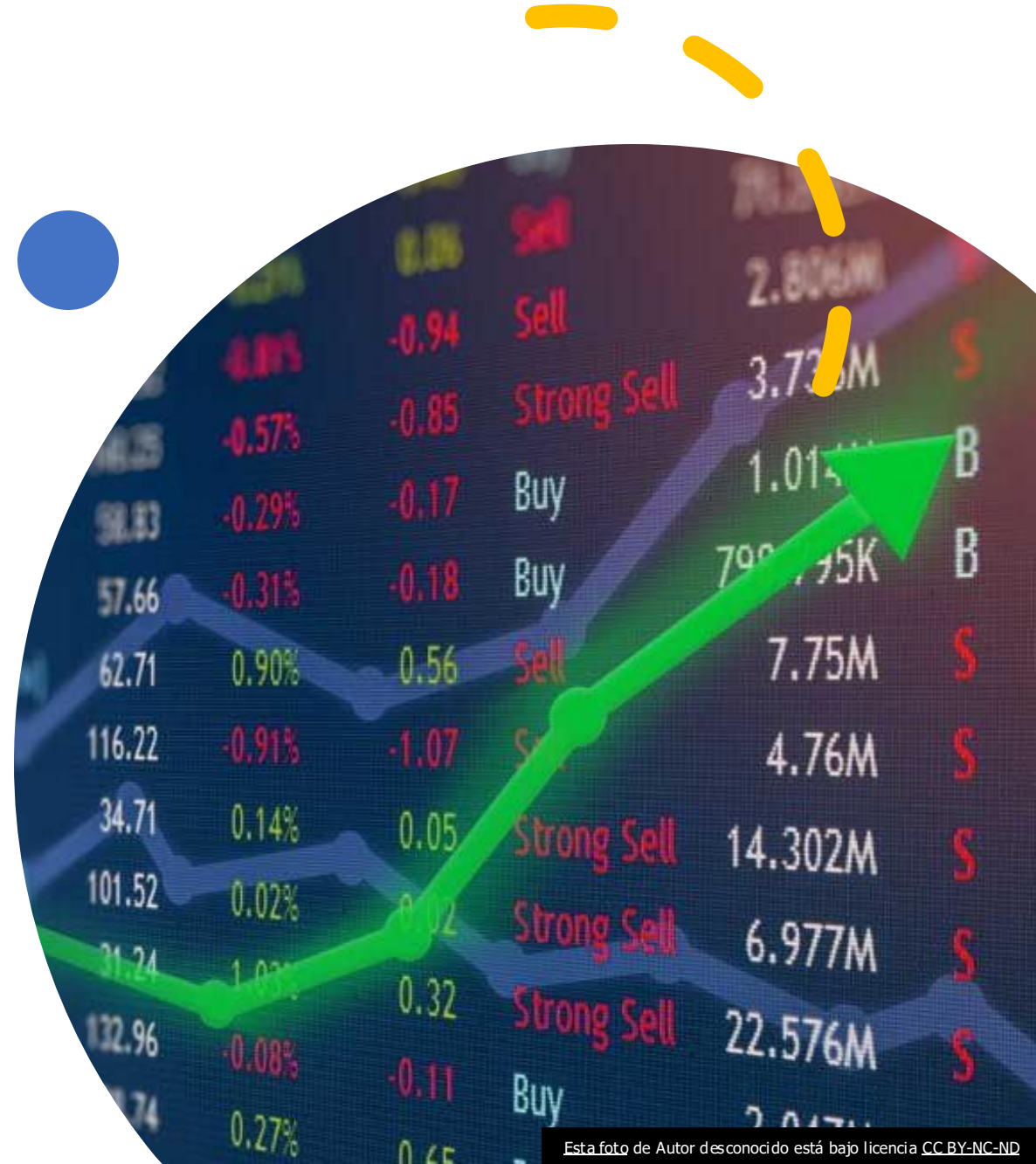
No tiene que ser extremadamente profundo

No implica que se le privará de explicaciones

¿Cómo influye el conocimiento de dominio?

Un sistema puede necesitar distintas cosas en base a cómo va a funcionar:

- Desarrollo de un sistema de manejo de cartera para un Hedge Fund
- Desarrolle un sistema capaz de administrar y ejercer opciones, así como mostrarles a los clientes sus carteras de acciones.



¿Cómo influye el conocimiento de dominio?

¿Qué es un *stock*?

¿Qué significa una *Call*?

¿Cómo se ejercen las opciones?

¿Qué es un futuro?

¿Qué es un contrato aleatorio?

¿Cómo influye el conocimiento de dominio?

¿Qué es un *stock*?

¿Qué significa una *Call*?

¿Cómo se ejercen las opciones?

¿Qué es un futuro?

¿Qué es un contrato aleatorio?

**Conocer todo esto es
clave para poder
comunicar todas las
áreas y ofrecer un diseño
apropiado**

¿Cómo podemos medir lo que diseñemos?

Veamos algunas definiciones que nos ayudarán en la implementación

Cohesión

- Partes que deben ir juntas en un mismo módulo tienen buena cohesión
- Interactúan idealmente mediante campos comunes o llamadas relacionadas
- Buena cohesión: librería numpy
- Poca cohesión: librería String, io

```
text = ""
```

```
text.
```

```
capitalize
```

```
casefold
```

```
center
```

```
count
```

```
encode
```

```
endswith
```

```
expandtabs
```

```
find
```

```
format
```

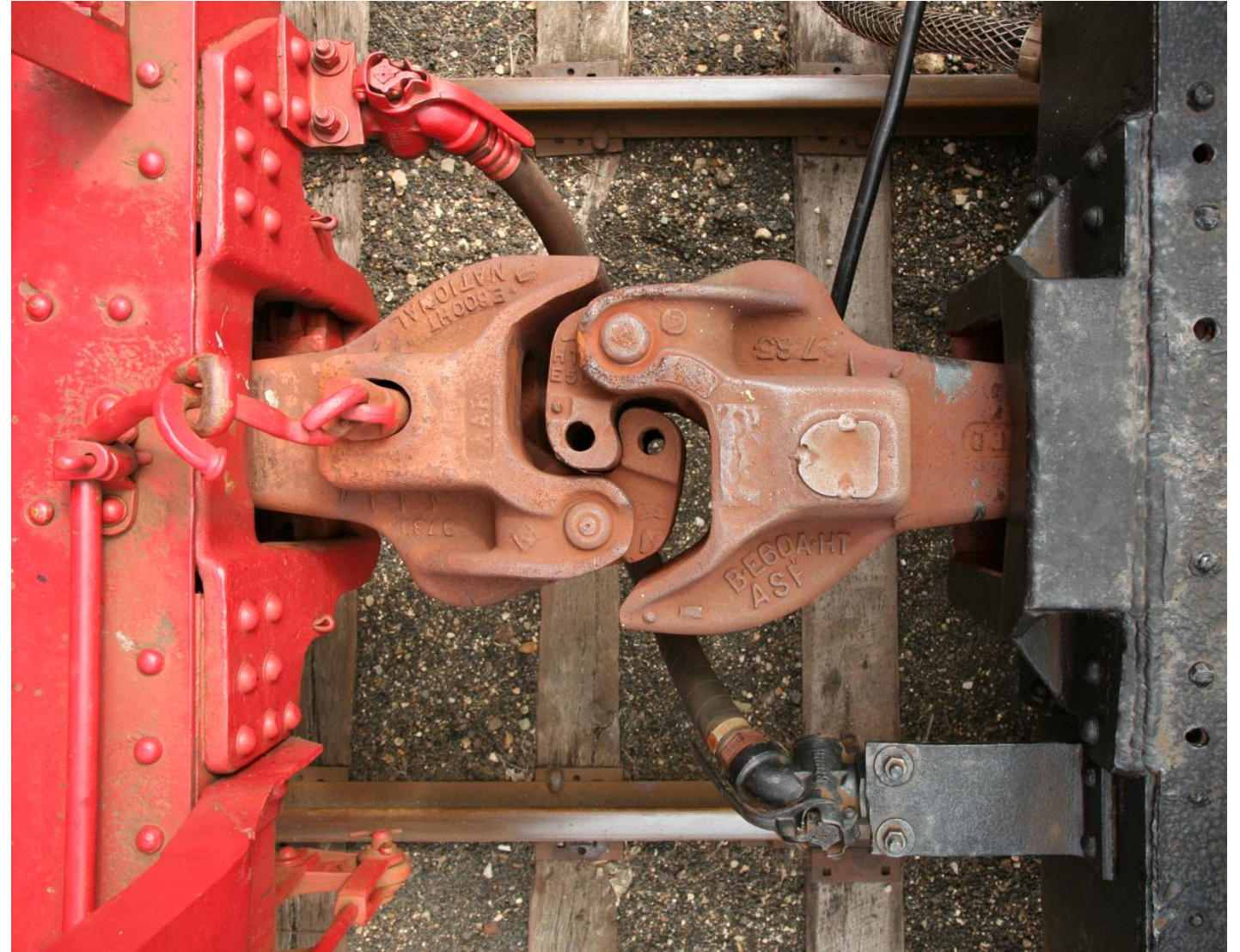
```
format_map
```

```
index
```

```
isalnum
```

Acoplamiento

- Grado de interdependencia de dos o más componentes de software
- Propiedad indeseable en muchos sistemas
- Mal necesario, se puede reducir con refactoring y patrones de diseño



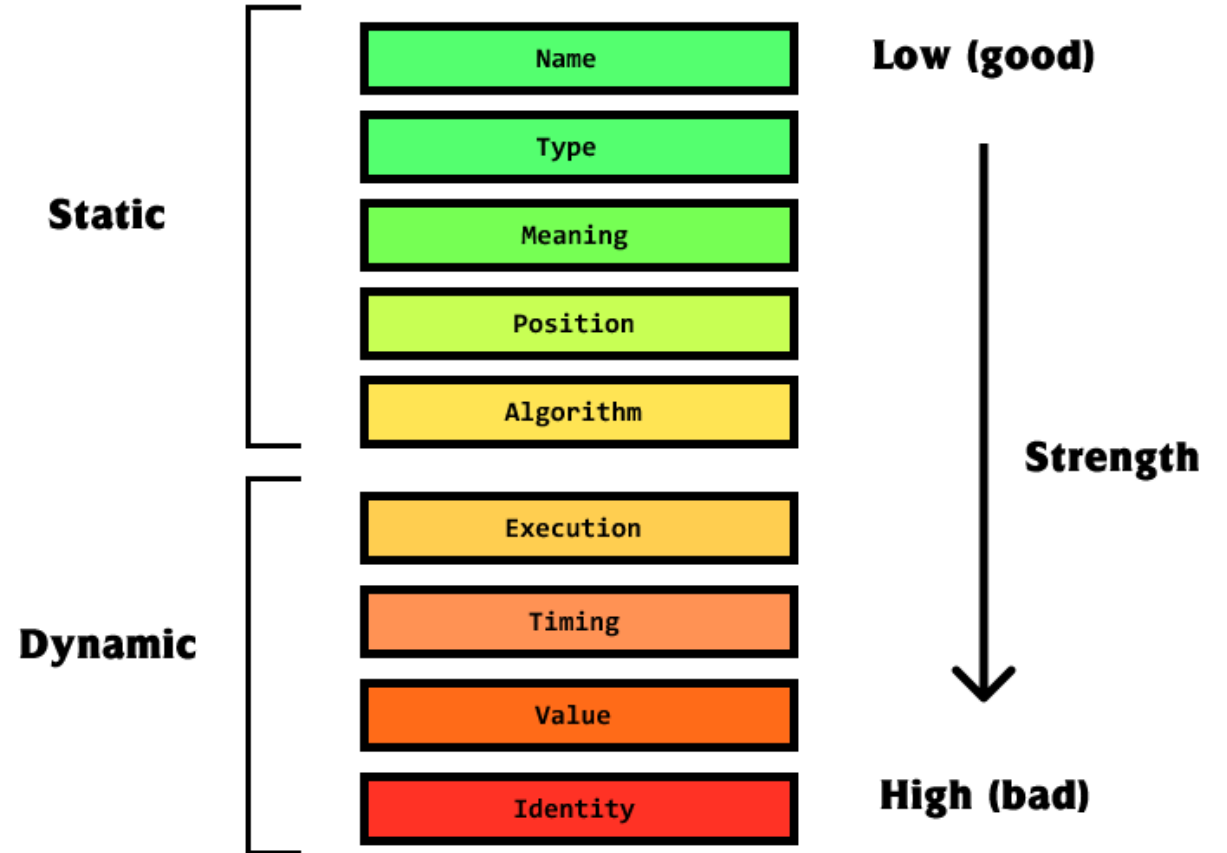
Sobre el acoplamiento

Tipo	Qué es
Contenido	Dependencia de implementación interna
Común	Uso de propiedades o variables entre módulos
Datos	Paso de datos específicos como parámetros
Control	Componentes que se controlan entre ellos
Stamp/estructura de datos	Operan sobre estructuras de datos comunes (e.g. db)

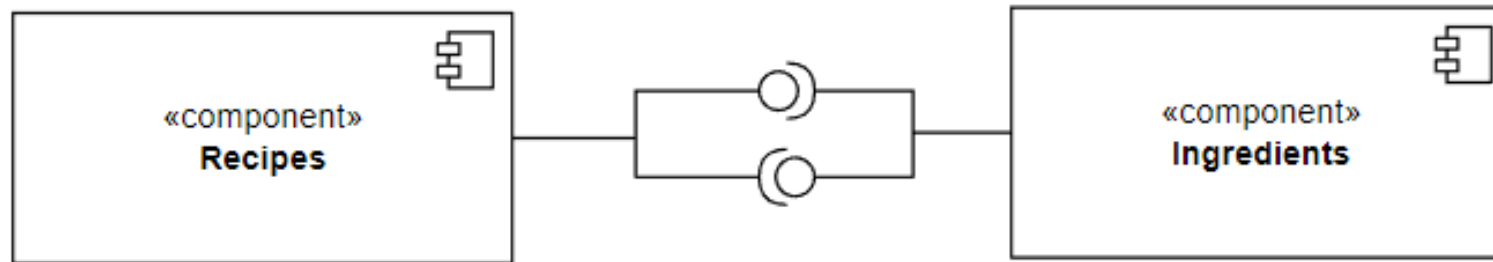
Connascence

"Two components are connascence if a change in one would require the other to be modified in order to maintain the overall correctness of the system"

Meilir Page-Jones



Connascence: Ejemplo en Aplicación de recetas



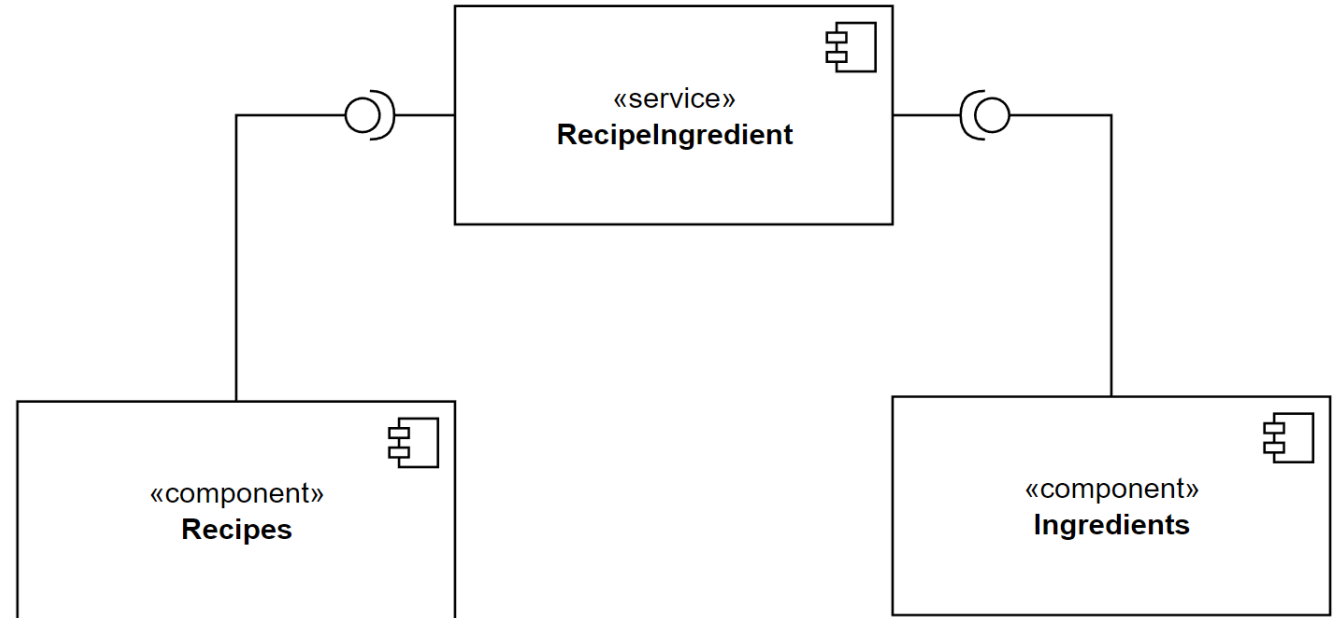
Una receta se ejecuta llamando a sus ingredientes, mientras que los ingredientes pueden llamar a su receta al momento de ejecutarse

¿De qué tipo es su *connascence*?

¿Cómo podemos mejorar esto?

Connascence

- ¿En verdad queríamos hacerlo?
- Pros: ya no se destruye todo
- Cons: Más código que mantener

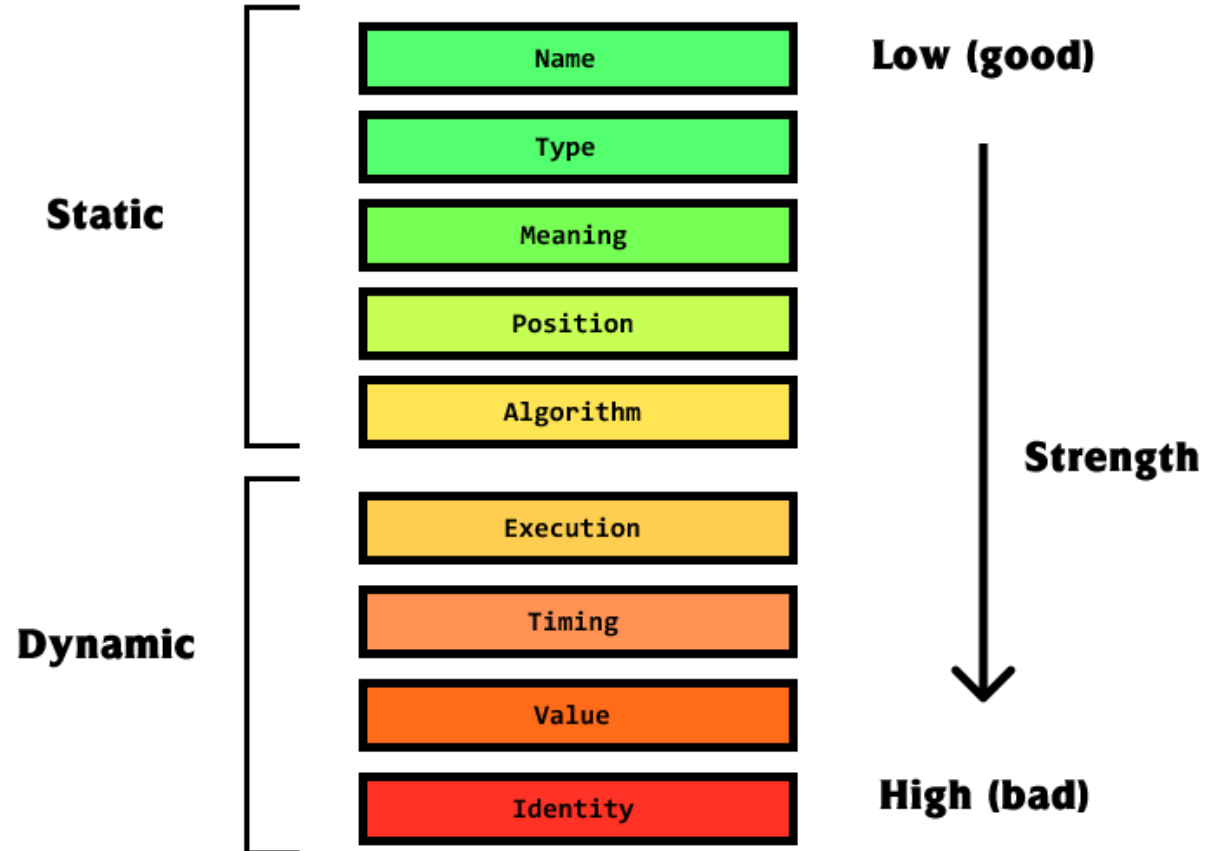


Connascence

- Convertir formas fuertes de *connascence* en formas más débiles de *connascence* – *Rule of Degree*
- Mientras aumenta la distancia de los elementos del software, queremos usar un *connascence* más débil - *Rule of Locality*

Jim Weirich

Buscamos un *connascence* más "deseable"

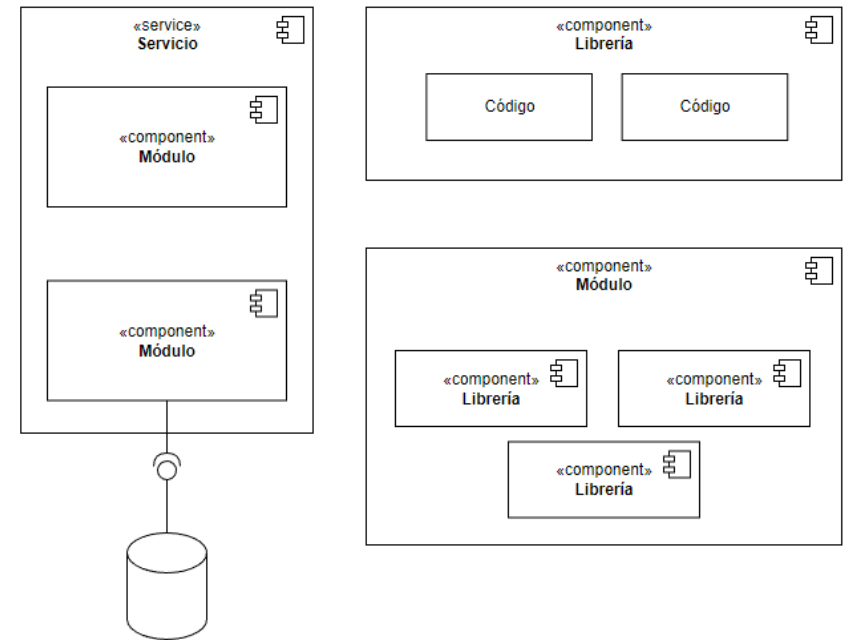


¿Cómo se representa lo que diseñemos?

Los conceptos que usaremos para entender el *software*

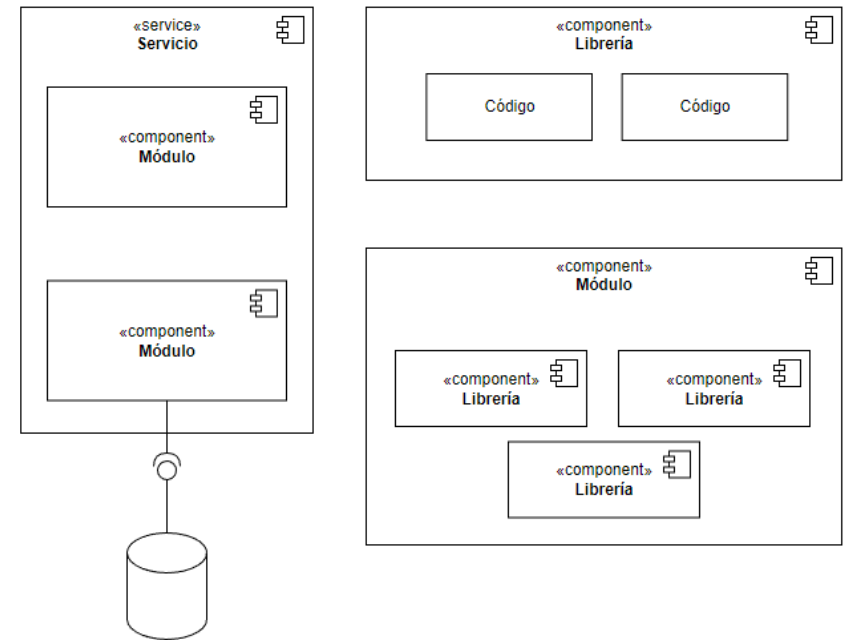
Componente

- Encapsula un subconjunto de funcionalidad y/o estado de un sistema
- Expone contratos/interfaces explícitos
- Altamente cohesionado, con “*connascence*” interna de diversos grados



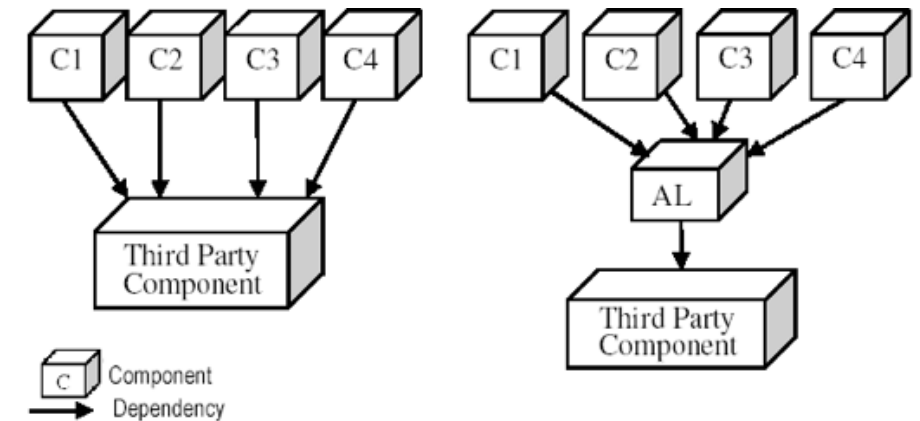
Componente

- Agrupaciones lógicas de software
 - Librerías
 - Módulos
 - SubSistemas
 - Sistemas



Unidad o quanta arquitectónica

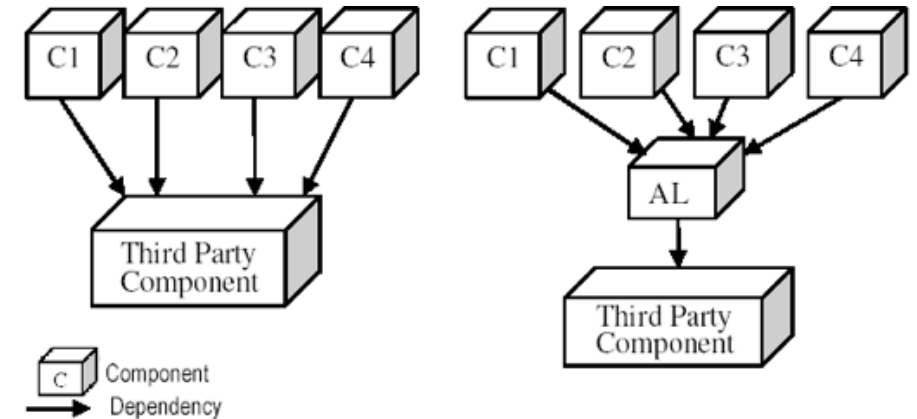
- "Architectural quantum is an *independently deployable component* with high functional cohesion"
 - Neal Ford, Rebecca Parsons and Patrick Kua-
- No olvidar que es una medida o una unidad y no la descripción de un componente o un set de reglas para la misma
- Relacionado a "*bounded context*" que veremos hacia el final del semestre



Unidad o quanta arquitectónica

A considerar:

- Distinguir varias quantas nos puede **ayudar a mejorar la resiliencia de nuestro sistema**
 - Se cae una parte, las **otras quantas son independientes** y deberán de seguir funcionando
- El concepto de quantas nos ayuda a identificar puntos vulnerables del sistema



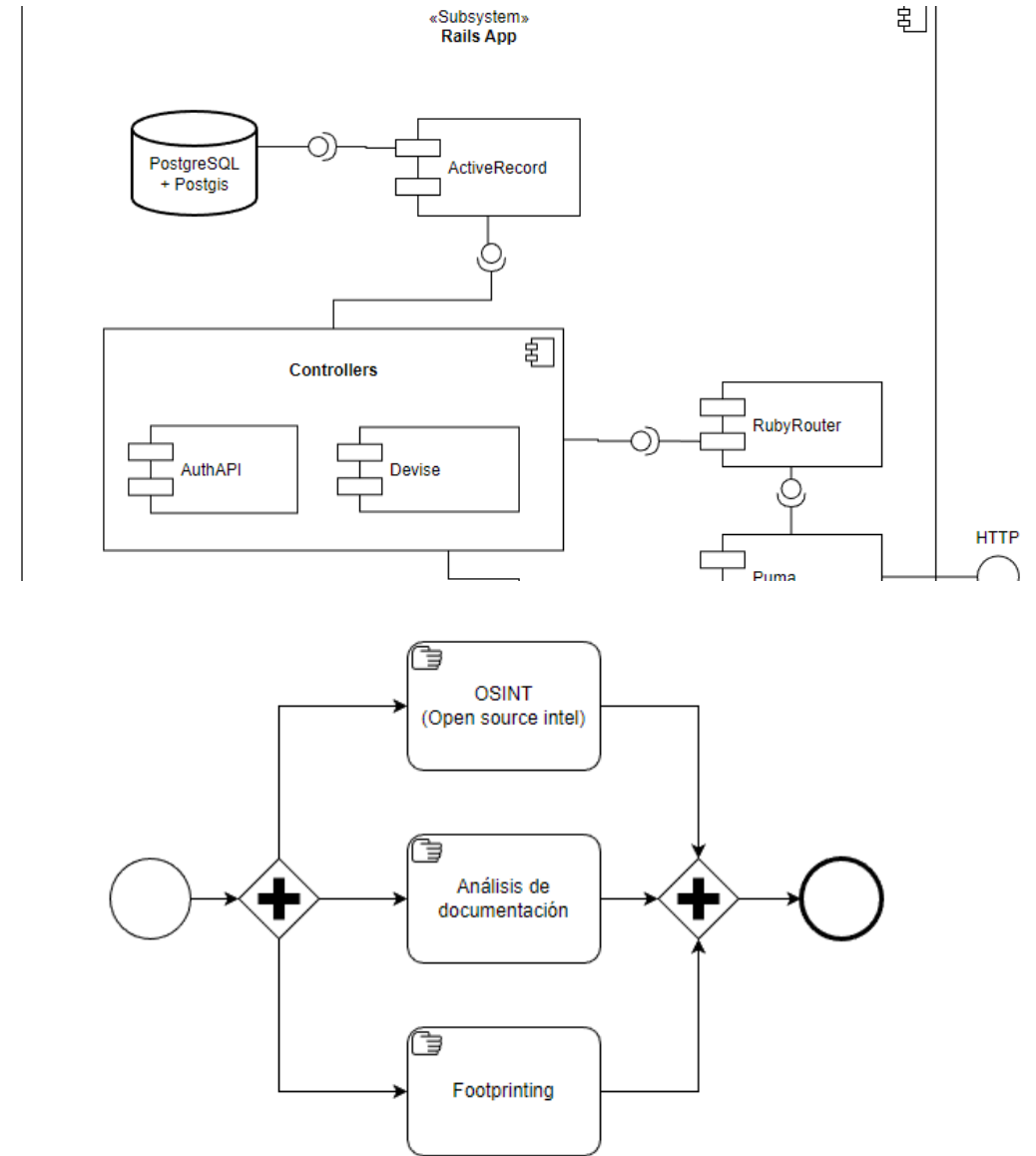
Documentando la arquitectura

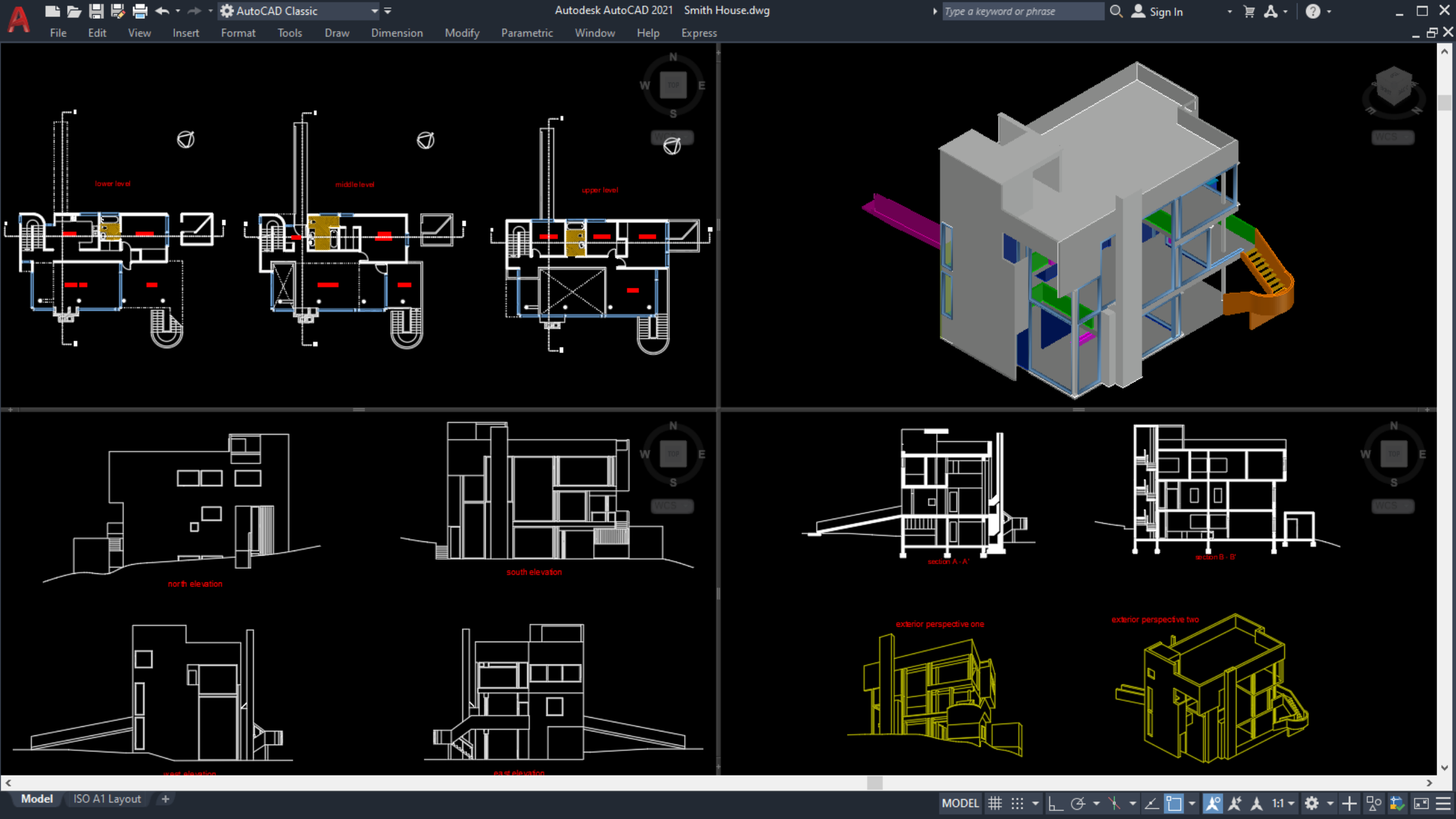
- Parte del rol de comunicador implica documentar a varios niveles:
 - Código
 - Comentarios
 - Informes
 - Vistas
 - ADRs



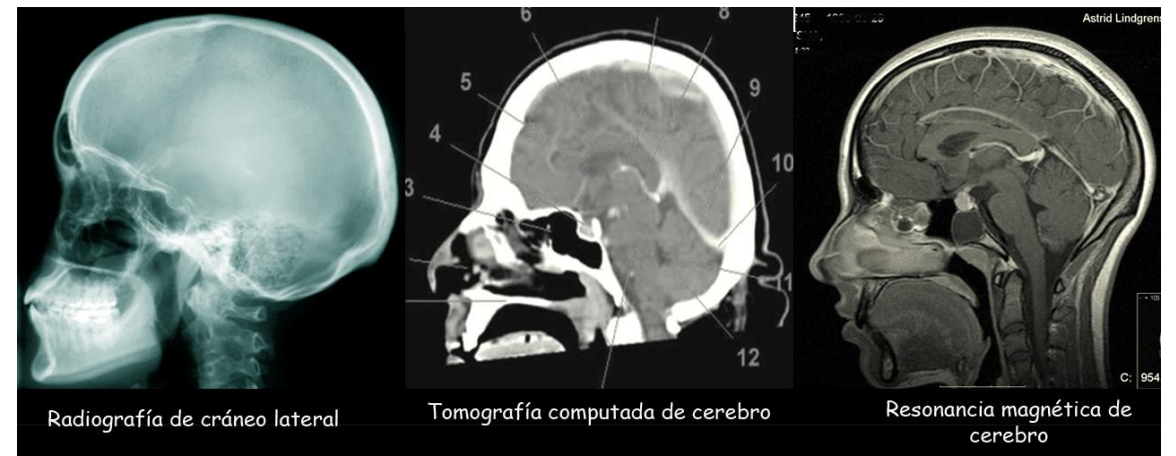
Vistas

- Colecciones de decisiones y hechos sobre un sistema, representados en un formato gráfico
- Ninguna vista posee toda la verdad: se requieren varios tipos





Ejemplo médico: cráneo



Vistas importantes

- 4+1
 - Es general, pero declara cosas importantes que ver
- UML
 - Componentes
 - Secuencia
- C4
 - Más moderna pero no absoluta



Architecture Decision Record (ADR)

- An Architectural Decision (AD) is a justified software design choice that addresses a functional or non-functional requirement that is architecturally significant.

(<https://adr.github.io/>)

Title

Status

What is the status, such as proposed, accepted, rejected, deprecated, superseded, etc.?

Context

What is the issue that we're seeing that is motivating this decision or change?

Decision

What is the change that we're proposing and/or doing?

Consequences

What becomes easier or more difficult to do because of this change?



Bibliografías

- Fundamentals of Software Architecture - Richards & Ford
 - Capítulos: 1, 2 y 3